

Emanation Mechanics — Architecture Review

Executive Summary

`MagicCircleEffectData.cs` is **highly Magic Circle-specific** but contains several **reusable patterns** that can be extracted into a base class. The GameManager tracking is also specific (`_activeMagicCircles`) but follows a pattern easily replicated or generalized. **Refactoring is recommended before implementing the next emanation** to avoid copy-paste proliferation.

1. Current Architecture Analysis

MagicCircleEffectData.cs — What's Generic vs Specific

Component	Reusable?	Notes
<code>CenterCreature</code>	✓ Generic	All emanations center on a creature
<code>RadiusSquares</code> / <code>RadiusFeet</code>	✓ Generic	All emanations have a radius
<code>RemainingRounds</code>	✓ Generic	All emanations have duration
<code>CasterLevel</code>	✓ Generic	Many emanations need CL for checks
<code>SourceSpellId</code> / <code>CasterName</code>	✓ Generic	Tracking/logging
<code>IsCreatureInArea()</code>	✓ Generic	Chebyshev distance check — universal for grid-based emanations
<code>GetCreaturesInArea()</code>	✓ Generic	Finding who's in the area — all emanations need this
<code>Tick()</code>	✓ Generic	Duration countdown — universal
<code>GetDurationDisplay()</code>	✓ Generic	UI display — universal
<code>WardedAlignment</code>	✗ Magic Circle only	Alignment-specific
<code>IsAttackerOfWardedAlignment()</code>	✗ Magic Circle only	Alignment checking
<code>GetSpellName()</code>	⚠ Override pattern	Each emanation would have its own name logic

Verdict: ~75% of the class is generic emanation logic. Only `WardedAlignment` and `IsAttackerOfWardedAlignment()` are alignment-specific.

GameManager.cs — Tracking System

The tracking in GameManager follows this pattern for Magic Circle:

```
// Storage
private readonly List<MagicCircleEffectData> _activeMagicCircles = new();

// CRUD
public void RegisterMagicCircle(MagicCircleEffectData data) // Add (replaces existing on same creature)
public void RemoveMagicCircle(CharacterController center) // Remove
public void TickMagicCircles() // Duration tick
public MagicCircleEffectData GetMagicCircleOnCreature(...) // Query single
public List<MagicCircleEffectData> GetActiveMagicCircles() // Query all
public bool IsInAnyMagicCircle(CharacterController creature) // Membership check

// Benefit resolution
public AlignmentProtectionBenefits GetMagicCircleBenefitsAgainst(...) // Specific to alignment
public bool IsProtectedByMagicCircle(...) // Specific to alignment
```

This is all Magic Circle-specific — each new emanation would need its own set of CRUD/query methods unless we generalize.

AlignmentProtectionBenefits Integration

The benefits are consumed via `AlignmentProtectionRules.GetBenefitsAgainst()` which:

1. Checks direct Protection from Alignment buffs on the target
2. Checks Magic Circle area effects via `GameManager.GetMagicCircleBenefitsAgainst()`
3. Takes the best (non-stacking per PHB rules)

This integration is **entirely alignment-specific** — other emanations (Bless, Prayer, auras) would plug into different stat pipelines.

2. Reusable Components (Extract to Base Class)

```

/// <summary>
/// Base class for all emanation effects that move with a center creature.
/// Handles area tracking, distance checks, duration, and creature membership.
/// </summary>
[System.Serializable]
public abstract class EmanationEffectData
{
    // — Core Properties (from MagicCircleEffectData) —
    [System.NonSerialized] public CharacterController CenterCreature;
    public int CasterLevel;
    public int RemainingRounds;
    public int RadiusSquares;
    public float RadiusFeet;
    public string SourceSpellId;
    public string CasterName;

    // — Abstract: each emanation defines its own name/effects —
    public abstract string GetSpellName();

    /// <summary>Override to apply/remove effects when a creature enters/exits the
    area.</summary>
    public virtual void OnCreatureEntersArea(CharacterController creature) { }
    public virtual void OnCreatureLeavesArea(CharacterController creature) { }

    // — Reusable Methods (identical to current MagicCircleEffectData) —
    public bool IsCreatureInArea(CharacterController creature)
    { /* ... same Chebyshev logic ... */ }
    public List<CharacterController> GetCreaturesInArea(List<CharacterController>
all) { /* ... same ... */ }
    public bool Tick() { /* ... same ... */ }
    public string GetDurationDisplay() { /* ... same ... */ }
}

```

Then MagicCircleEffectData becomes:

```

[System.Serializable]
public class MagicCircleEffectData : EmanationEffectData
{
    public AlignmentProtectionType WardedAlignment = AlignmentProtectionType.None;

    public bool IsAttackerOfWardedAlignment(Alignment attackerAlignment)
        => AlignmentProtectionRules.Matches(WardedAlignment, attackerAlignment);

    public override string GetSpellName() { /* existing switch logic */ }
}

```

3. Magic Circle-Specific Components (Cannot Reuse)

These are tightly coupled to alignment protection and would NOT be shared:

- `AlignmentProtectionType WardedAlignment` — unique to Protection/Circle spells
- `IsAttackerOfWardedAlignment()` — alignment matching

- **AlignmentProtectionBenefits struct** — deflection AC, resistance saves, mental control immunity, summoned barrier
- **GetMagicCircleBenefitsAgainst()** in GameManager — alignment-specific benefit resolution
- **IsProtectedByMagicCircle()** in GameManager — mental control suppression check
- **Integration in AlignmentProtectionRules.GetBenefitsAgainst()** — stacking/non-stacking logic with Protection from Alignment

4. GameManager Generalization Options

Option A: Generic Emanation Registry (Recommended)

```
// Replace individual lists with a single registry:
private readonly List<EmanationEffectData> _activeEmanations = new();

public void RegisterEmanation(EmanationEffectData data) { /* ... */ }
public void RemoveEmanation(CharacterController center) { /* ... */ }
public void TickEmanations() { /* ticks all, removes expired */ }

// Type-specific queries:
public T GetEmanationOnCreature<T>(CharacterController creature) where T : EmanationEffectData { /* ... */ }
public List<T> GetActiveEmanationsOfType<T>() where T : EmanationEffectData { /* ... */ }
public bool IsInAnyEmanation<T>(CharacterController creature) where T : EmanationEffectData { /* ... */ }
```

Pros: Single tracking system, DRY, easy to add new emanation types.

Cons: Generic queries slightly more complex; Magic Circle-specific methods need to cast.

Option B: Keep Separate Lists Per Emanation Type

```
private readonly List<MagicCircleEffectData> _activeMagicCircles = new();
private readonly List<BlessEffectData> _activeBlessEffects = new();
private readonly List<PrayerEffectData> _activePrayerEffects = new();
// ... one per type
```

Pros: Simple, type-safe, no casting needed.

Cons: Duplicated CRUD methods per type; GameManager grows linearly with each emanation.

Recommendation: Option A — the CRUD pattern is identical across emanation types and should be shared.

5. Future Emanations and How They'd Extend

Bless (1st-level Cleric)

- **Type:** 50-ft burst (fixed area, NOT emanation) — does NOT move with caster
- **Effect:** +1 morale bonus to attack rolls, +1 morale bonus on saves vs fear
- **Duration:** 1 min/level

- **Note:** This is actually a BURST, not an emanation. It affects all allies at cast time and doesn't track area membership. Would NOT use emanation base class. Uses simple buff application.

Prayer (3rd-level Cleric)

- **Type:** 40-ft radius emanation centered on caster
- **Effect:** +1 luck bonus to attack, damage, saves, skill checks for allies; -1 for enemies
- **Duration:** 1 round/level
- **Would extend:** `EmanationEffectData` with `OnCreatureEntersArea` / `OnCreatureLeavesArea` to apply/remove luck modifiers

```
public class PrayerEffectData : EmanationEffectData
{
    public override string GetSpellName() => "Prayer";

    // Needs: track who's ally vs enemy
    // Apply +1 luck to allies, -1 luck to enemies within area
    // Remove when they leave or effect expires
}
```

Paladin Aura of Courage (Su)

- **Type:** 10-ft radius emanation centered on Paladin
- **Effect:** Immunity to fear for allies; +4 morale bonus on saves vs fear
- **Duration:** Permanent while Paladin is conscious
- **Would extend:** `EmanationEffectData` with `RemainingRounds = int.MaxValue` (permanent)

```
public class AuraOfCourageEffectData : EmanationEffectData
{
    public override string GetSpellName() => "Aura of Courage";
    // Apply fear immunity + save bonus to allies in area
    // Remove when they leave area or paladin falls unconscious
}
```

Bard Inspire Courage (Su)

- **Type:** 30-ft radius emanation centered on Bard
- **Effect:** +1 morale bonus to saves vs charm/fear, +1 morale to attack/damage (increases at higher levels)
- **Duration:** As long as Bard sings + 5 rounds after
- **Would extend:** `EmanationEffectData` with special duration logic

Consecrate/Desecrate (2nd-level Cleric)

- **Type:** 20-ft radius fixed emanation (does NOT move — centered on a point)
- **Note:** This is emanation from a POINT, not a creature. Would need `CenterPosition` as alternative to `CenterCreature`.

```
// Would need base class to support:
[System.NonSerialized] public CharacterController CenterCreature; // null for fixed-point
public Vector2Int? CenterPosition; // for point-centered emanations
```

6. Recommended Refactoring Steps

Phase 1: Extract Base Class (Do before next emanation)

1. Create `EmanationEffectData.cs` with shared properties and methods
2. Make `MagicCircleEffectData` inherit from it
3. Add `CenterPosition` support for fixed-point emanations (Consecrate/Desecrate)
4. Verify all tests still pass

Phase 2: Generalize GameManager Tracking

1. Replace `_activeMagicCircles` with `_activeEmanations` list
2. Create generic `RegisterEmanation<T>()`, `RemoveEmanation()`, `TickEmanations()`
3. Keep Magic Circle-specific query methods but have them filter by type
4. Add `OnCreatureEntersArea` / `OnCreatureLeavesArea` callbacks in tick loop

Phase 3: Implement Next Emanation Using New System

1. Prayer or Paladin Aura of Courage are good candidates (both are true emanations)
2. Should “just work” with the base class + registration system

7. Key Design Decisions

Decision	Recommendation	Rationale
Base class vs interface	Abstract base class	Too much shared implementation for an interface
Generic list vs typed lists	Generic <code>List<EmanationEffectData></code>	Avoids CRUD duplication
Enter/exit callbacks	Virtual methods on base class	Cleanest way to handle effect application
Fixed-point emanations	<code>CenterPosition</code> as nullable field	Supports Consecrate/Desecrate without separate hierarchy
Ally/enemy detection	Pass team info to emanation methods	Emanation shouldn't assume PC/NPC structure
Stacking rules	Each emanation type handles own stacking	Too varied for a generic system (morale doesn't stack, luck doesn't stack, etc.)

8. What Does NOT Need Refactoring

- **AlignmentProtectionBenefits** and its integration in combat — this is correctly specific to alignment spells
 - **The Chebyshev distance check** — already correct for grid emanations, just needs to be in base class
 - **Duration display** — generic enough as-is
 - **The combat integration points** (AC calculation, save calculation) — these correctly query specific effect types and should continue to do so
-

Summary

Current state: MagicCircleEffectData is a solid implementation with ~75% reusable code. The GameManager tracking is clean but specific.

Effort to generalize: Low-medium. Extract base class (~30 min), generalize GameManager tracking (~1 hour), write first additional emanation to validate (~1-2 hours).

When to refactor: Before implementing the next emanation spell. Don't refactor speculatively — wait until Prayer, Paladin auras, or Bard songs are on the roadmap, then extract the base class as the first step.